Franklin Open 10 (2025) 100200

Contents lists available at ScienceDirect

Franklin Open

journal homepage: www.elsevier.com/locate/fraope

Enhanced hybrid facial emotion detection & classification

Naim Ajlouni [a,*], Adem Özyavaş [a], Firas Ajlouni [b], Faruk Takaoğlu [c], Mustafa Takaoğlu [c]

[a] Istanbul Atlas University, Türkiye
[b] Lancashire College of Further Education, UK
[c] The Scientific and Technological Research Council of Turkey, TÜBİTAK-BİLGEM, Türkiye

ARTICLE INFO

Keywords:
Emotion detection
Convolutional neural network
Facial landmarks
Genetic algorithms
Active shape models
Fully Adaptive SGD

ABSTRACT

Emotion Recognition (ER) presents a significant challenge in pattern recognition and is crucial for various Artificial Intelligence (AI) applications, from monitoring children with autism to enhancing video games and human-computer interactions. Facial features are essential for discerning human emotions, motivating this study to improve Facial Emotion detection accuracy, vital for real-time applications. This research explores the use of both full facial features and facial landmarks, suggesting that landmarks offer clearer emotional cues. A Convolutional Neural Network (CNN) is employed for feature extraction, optimized using techniques like Stochastic Gradient Descent (SGD). This study combines upper facial landmarks with full facial features to enhance detection accuracy. An adaptive Genetic Algorithm (GA) optimizes the CNN structure. The dual-input neural network architecture processes a full 48 × 48 facial image through a CNN and upper facial features through another input, which is then concatenated and fed into dense layers with L2 regularization. The model is trained and evaluated using various optimizers to find the best configuration, with performance metrics plotted and compared. This method allows detailed pixel information and focused landmark features to improve emotion detection accuracy. Datasets such as FER2013 and Extended Cohn-Kanade (CK+) are used for training and testing. Results show significant accuracy improvement with the proposed method, further enhanced by a highly optimized model structure and an adaptive SGD optimizer. Integrating adaptive learning and momentum terms minimizes model loss and speeds up convergence.

1. Introduction

Facial expression recognition (FER) serves as a critical component of human-computer interaction, enabling systems to interpret emotional cues from human facial features, and has significant applications in areas like lie detection, e-learning, robotics, and customer behavior analysis, particularly with the rise of the Internet of Things (IoT) (Alarcao & Fonseca, 2019; Li & Deng, 2020) [1,2]. While traditional FER methods relied heavily on human interpretation, advancements in machine learning (ML) and deep learning (DL) have fundamentally transformed this field, enabling automated FER with remarkable accuracy (Martinez et al., 2017) [3]. Existing FER techniques typically employ either dynamic approaches, which analyze expressions across sequences of frames, or static methods, which assess individual frames and involve steps like image enhancement, face acquisition, feature extraction, and emotion classification (Li & Deng, 2019; Zhao et al., 2019) [4,5]. Despite these advances, real-world applications of FER models remain challenging due to pose variations, lighting inconsistencies, and facial occlusions, which can impede system accuracy (Zhao et al., 2019) [5]. This study addresses these limitations by integrating Genetic Algorithms (GA) to optimize Convolutional Neural Network (CNN) architectures for better feature extraction and employing an Adaptive Stochastic Gradient Descent (SGD) optimizer to enhance convergence, reduce loss, and achieve higher classification accuracy. Through this approach, we aim to advance FER capabilities in complex, real-world settings, evaluating the optimized CNN model's performance against a conventional CNN to underscore the benefits of our hybrid method.

The structure of the paper is as follows: Section 2 provides an overview of the datasets used in the study. Section 3 discusses related work, examining prior approaches to face detection and feature extraction. Section 4 delves into the algorithms for face detection and methods for feature extraction. Section 5 explains the methodology behind the convolutional neural network, while Section 6 covers techniques for detecting facial landmarks. The proposed method is outlined in Section 7, with the research contributions detailed in Section 8. Section 9 presents the experimental analysis and discussion. Finally, Section 10

* Corresponding author.
E-mail address: naim.ajlouni@atlas.edu.tr (N. Ajlouni).

https://doi.org/10.1016/j.fraope.2024.100200
Received 29 June 2024; Received in revised form 26 October 2024; Accepted 10 December 2024
Available online 16 December 2024

concludes the paper with summarizing remarks.

2. Datasets

Two different datasets s will be used to train, test, and verify the proposed methods in this study. The datasets are on FER2013 (https://www.kaggle.com/msambare/ ferfer2013) and the Extended Cohn-Kanad (CK+) whis available in Kaggle (https://www.kaggle.com/datasets/davilsena/ckdataset).

2.1. Fer2013 dataset

The dataset consists of 48 × 48 pixel grayscale images of faces, totaling 32,298 images. These images are categorized into seven folders based on different expressions: angry (4893 images), disgust (547 images), fear (5121 images), happy (8989 images), neutral (6198 images), sad (6077 images), and surprise (4002 images). A sample of the dataset images is shown in Fig. 1.

2.2. Extended Cohn–Kanade (CK+) face dataset

The CK+ dataset comprises 820 grayscale images, each sized at 48 × 48 pixels. These images are categorized into folders representing different emotional expressions: Anger (45 images), Disgust (59 images), Fear (25 images), Happiness (69 images), Sadness (28 images), Surprise (83 images), Neutral (593 images), and Contempt (18 images). Fig. 2 illustrates a sample of these dataset images.

3. Related work

The human face is a vital communicator of emotions and expressions, including anger, despair, happiness, love, dislike, fear, and surprise. At a first glance, facial features can reveal a person's identity, gender, age, emotions, and race. Facial expressions also play a key role during social interactions, offering communicative clues about behavior, engagement in discussions, and understanding of the given information (Huang, Chen, Lv, & Wang, 2019) [6]. This information is vital for effective communication, providing insights into a person's interest and attention level. With the growing integration of Internet-of-Things (IoT)-enabled environments and advances in machine learning (ML) and deep learning (DL), facial expression recognition (FER) has become more prevalent in applications such as lie detectors, e-learning platforms, robotics, and customer behavior tracking. Facial expressions can communicate without words (Harper, Wiens, & Matarazzo, 1978) [7], making FER systems a valuable tool for enhancing human-computer interaction.

Emotion recognition, a process that detects human feelings based on speech, activity, or facial expressions, has evolved significantly. Speech-based systems commonly use natural language processing (NLP) techniques, whereas facial expression-based systems require image-based methods for emotion analysis. These methods can be used to analyze instructor efficiency in e-learning or monitor customer interest during product promotions. Traditionally, humans excelled at interpreting facial expressions, but ML and DL techniques have now revolutionized this domain, enabling automatic FER with high accuracy.

FER methods are generally categorized as dynamic or static. Dynamic methods analyze sequences of facial expressions over time (Byeon & Kwak*, 2014) [8], while static methods process single frames (P. Liu, Han, Meng, & Tong, 2014; Shan, Gong, & McOwan, 2009) [9,10]. Regardless of the method, FER involves four primary steps: image enhancement, face acquisition (Chen, Wong, & Chiu, 2011) [11], facial feature extraction, and emotion classification (Lakshmi & Ponnusamy, 2021) [12]. Various computer vision (CV) techniques such as wavelet transformation and filtering are employed to enhance noisy images. Next, face acquisition extracts the face from input images using statistical or template-based methods. Feature extraction captures both global and local characteristics, including motion, shape, geometry, and texture (Agrawal, Dubey, & Jalal, 2014) [13], which are then used for emotion classification.

FER often relies on changes in facial features such as the forehead, nose, lips, and eyes. These features can be extracted using two major approaches: appearance-based and geometric-based. Appearance-based methods analyze pixel intensity and neighboring relations, while geometric-based methods utilize angles, areas, and distances between facial landmarks. These features are then grouped to classify emotions like anger, joy, or surprise. Researchers have proposed several algorithms that link facial features with emotions to enhance FER performance. For example, Valstar et al. (2012) [14] introduced the first comprehensive analysis of FER by dividing it into classification and action unit detection.

Various FER systems have been developed to improve performance in real-world applications. Abdat, Maaoui, & Pruski (2011) [15] proposed an ML-based system that extracts facial features using an anthropometric model and the Shi-and-Thomas method, while classifying expressions with Support Vector Machines (SVM). Similarly, Littlewort et al. (2011) [16] used a static detection approach combined with a SVM classifier to classify emotions, achieving an F1-score of 86 % on the CK+ dataset. Ghimire et al. (2017) [17] combined geometric- and appearance-based methods, achieving over 97 % accuracy using SVM for dimensionality reduction.

Other studies explored additional techniques such as the k-Nearest Neighbor (kNN) algorithm, random forests, decision trees, and deep neural networks. Tang et al. (2015) [18] evaluated teaching environments using kNN and ULGBPHS, achieving 96.7 % accuracy on the JAFFE dataset. Nguyen et al. (2017) [19] compared random forests,

Fig. 1. Sample of FER Dataset images.

Fig. 2. CK+ Dataset sample.

SVM, and decision trees for real-time FER, with SVM outperforming others on accuracy. Recent models, such as those combining Local Binary Patterns (LBP) and Histogram of Oriented Gradients (HoG) with SVM, have achieved notable results, including 97.66 % accuracy on the JAFFE and CK+ datasets.

Several studies have also applied scale-invariant feature transform (SIFT) techniques for FER. Ce Liu, Yuen, & Torralba (2011) [20] introduced a SIFT flow-based method that aligns expressive facial images to extract features. Revina & Sam Emmanuel (2019) [21] expanded on this by integrating scatter local-directional pattern (SLDP) features, achieving 96 % accuracy on JAFFE and CK datasets using a MultiSVNN with W-GOA.

Neural networks (NN) have also demonstrated impressive results in FER tasks. Halder et al. (2016) [22] developed an NN-based model that calculates facial feature similarities for classification, while Salmam et al. (2018) [23] used a single-layer NN to achieve high accuracy on multiple datasets. Other research explored hybrid models, such as Dinakaran & Ashokkrishna (2020) [24], who combined features like skin texture and geometry for FER, achieving 98.4 % accuracy. CNN models have also become popular, with Hassouneh, Mutawa, & Murugappan (2020) [25] achieving 99.81 % accuracy using CNN combined with EEG signals. A ResNet-inspired CNN model by Gupta, Arunachalam, & Balakrishnan (2020) [26] achieved 64.4 % accuracy with YCbCr images.

FER continues to evolve, with deep learning frameworks providing more powerful tools for analyzing emotions. For example, a recent GAN-based framework proposed by Peng et al. (2020) [27] improved performance on imbalanced datasets, achieving 73 % accuracy on FER2013 and CK+ datasets. Other studies have explored CNN-based models for multi-label learning environments (Li & Deng, 2019) [28], while Gogate et al. (2020) [29] evaluated gender classification alongside FER, achieving 67 % accuracy on emotion recognition.

Micro-facial expressions are brief, involuntary movements of the face that can reveal concealed emotions, often lasting only a fraction of a second. These subtle expressions provide deeper insights into human emotions, particularly in situations where individuals may attempt to hide their true feelings. This makes them invaluable in fields such as psychology, security, and customer behavior analysis. Recent advancements have aimed at improving the recognition of micro-expressions to enhance the accuracy of facial recognition systems. For instance, S. Ashok Kumar (2014) [30] developed a method to automatically analyze facial expressions from images, accurately identifying the emotions depicted. Similarly, S.D. Lalitha (2019) [31] outlined key steps to enhance the performance of micro-facial expression recognition, including the use of Adaptive Homomorphic Filtering for face detection and rotation correction, followed by the extraction of micro-facial features to analyze appearance variations in test images through spatial analysis.

Despite numerous advances, FER systems still face challenges, particularly in real-world applications where the accuracy of these systems can be hindered by variations in pose, lighting, or obstructions like glasses and facial hair. Further optimization is needed to improve their robustness and performance across diverse environments. This study aims to address these issues by using Genetic Algorithms (GA) to optimize the structure of Convolutional Neural Networks (CNNs) for better feature extraction. The CNN model is trained using an Adaptive Stochastic Gradient Descent (SGD) optimizer, which improves convergence, reduces loss, and enhances overall accuracy. The performance of this optimized CNN is compared to a conventional CNN with the same optimizer.

4. Face detection & feature extraction

Face detection is a crucial technology in image processing and computer vision, involving the identification of instances of objects such as human faces. Significant advancements in this field have been driven by deep learning techniques. One of the foundational methods in face detection is the Viola-Jones algorithm, introduced in 2001, which marked a major breakthrough in the technology.

The Viola-Jones algorithm is a powerful tool for real-time face detection, utilizing Haar-like features to identify faces efficiently. It operates through four primary steps: selecting Haar-like features, creating an integral image, conducting AdaBoost training, and building classifier cascades. Haar-like features leverage universal properties of human faces, such as the contrast between the eyes and surrounding areas or the brightness of the nose compared to the eyes. By comparing pixel values across different regions, the algorithm effectively identifies facial features.

The integral image data structure enhances computational efficiency by enabling rapid calculation of pixel value sums in rectangular regions, significantly reducing the operations needed. The algorithm's ability to combine multiple Haar-like features and utilize integral images allows for efficient real-time face detection, establishing it as a foundational technique in the field.

However, in our study, we did not utilize the Viola-Jones algorithm for face detection. Instead, we focused on more contemporary methods that leverage deep learning techniques for enhanced performance. This differentiation is crucial in understanding the context of our research

and the advancements it aims to contribute to the domain of facial emotion recognition.

4.1. Feature extraction

Feature extraction is crucial in image classification as it captures the content of images accurately. This paper compares several feature extraction techniques using different classifiers and evaluates their performance in image classification. The main feature extraction algorithms include:

1. Gabor Algorithm: The facial wrinkle is an essential feature for age estimation and emotion detection. Wrinkles, unique to each person, are extracted using Gabor filters with different scales and orientations applied to eleven skin areas. The magnitude of the resulting complex images is used to extract features [32].
2. LBP Algorithm: In 1994 Local Binary Pattern (LBP) algorithm was introduced, it is still been used for texture classification, facial analysis, motion analysis, and age classification. LBP detects microstructure patterns such as spots, lines, and edges by assigning a code to each pixel and comparing it with its neighbors [24].
3. LPQ Algorithm: The Local Phase Quantization (LPQ) operator, utilizes the fourier phase spectrum to achieve blur invariance property. It captures local phase information over a square neighborhood at each pixel position, serving as an effective image descriptor [33].
4. HOG Algorithm: Histograms of Oriented Gradients (HOG) are feature descriptors used to detect objects by extracting the magnitude and edge orientation of an image [17,29].

A feature fusion method is employed to extract features necessary for training selected optimizers. The proposed GA optimizes the CNN layer filters to achieve optimal results.

4.2. Feature fusion

In this method, the feature descriptors extract a limited number of information from the images to solve feature fusion problems. The feature-level fusion eliminates the redundant information resulting from the correlation of fused features [12]. Hence, global features and local features are combined. The next step is normalization, so the z-score normalization is used to normalize each feature.

$$f_i^{'} = \frac{j_i - \mu_i}{\sigma_i} \quad i = HOG,\ LBP,\ LPQ,\ \ldots\ldots \tag{1}$$

Where f_i is the ith feature vector, while μ_i and σ_i are the mean and standard deviation of the i^{th} feature, respectively, and $f_i^{'}$is the normalized feature vector. The feature fusion is constructed by interlinking the normalized features as follows:

$$f_{fusion} = \left(f_{LBP} f_{LPQ} f_{GW} f_{HOG}\right) = (C_1,\ \ldots, C_k, H_1,\ \ldots, H_b, w_1,\ \ldots,\ w_m) \tag{2}$$

Where f_{usion} is the hybrid features. Concatenating features increases the dimension of features, which leads to increased time consumption and increased memory usage.

5. Convolution Neural Network (CNN)

CNN is a Deep Learning model tailored to handle data with a grid-like structure, such as images. It is composed of various layers including convolutional, pooling, and fully connected layers. The convolutional and pooling layers are responsible for feature extraction, while the fully connected layer maps these features to output. The convolution layer executes a linear operation by applying a small grid of parameters called a kernel across the image to extract features. This makes CNNs particularly effective for image processing because they can identify features regardless of their position in the image. Optimizing these parameters, known as training, involves minimizing the error between the CNN's predictions and the actual labels using an optimization algorithm like backpropagation. After convolution, the outputs are passed through a nonlinear activation function, such as sigmoid or Rectified Linear Unit (ReLU), to enable the model to learn complex patterns. The pooling layer performs downsampling, to reduce dimensionality of feature maps which provides translation invariance to minor shifts and distortions. There are two types of pooling, max pooling and average pooling are the two main types, with max pooling being more common. Max pooling extracts the maximum value from each region of the feature map, thereby downsampling the features and decreasing the number of parameters for the following layers. For example, a max-pooling operation with a 2×2 filter and a stride of 2 reduces the feature map size by half.

A typical CNN architecture consists of multiple blocks of these layers, usually several convolutional layers followed by pooling layers, and finally fully connected layers. The process of transforming input data through these layers to produce an output is known as forward propagation. Although this description focuses on 2D-CNN operations, similar processes can be applied to three-dimensional (3D) CNNs.

6. Facial landmark

Facial landmark measurement theories are essential in various fields such as computer vision, biometrics, forensic science, and medical diagnostics. These theories focus on identifying key points on the face and using them for various applications like facial recognition, expression analysis, and reconstructive surgery. There exist a number of methods for facial landmark measurements including Anthropometric Landmarks, Geometric Feature-based Methods, Machine Learning and Deep Learning Methods, Machine Learning and Deep Learning Methods, and Statistical Shape Analysis. In this study, it is intended to utilize a Geometric Feature extraction method namely Active Shape Models (ASMs).

The ASM is said to be the more accurate and robust approach for facial landmark detection [35–38]. These models are particularly effective in capturing the geometric and appearance variations of facial features.

6.1. Active Shape Models (ASM)

ASM are based on statistical shape analysis and optimization techniques to accurately fit a model to observed data. In ASM, landmarks, which are distinct points present in all the considered images, such as the location of the eyes, are used to identify facial features. A set of landmarks forms a shape, and these shapes are represented as vectors of all the x-coordinates followed by all the y-coordinates of the points in the shape.

The alignment process involves matching one shape to another using a similarity transform to minimize the average Euclidean distance between corresponding shape points. The mean shape is then calculated from these aligned shapes. ASM uses this mean shape to search for landmarks, typically within a specific position and size as determined by a global face detector.

The ASM process iterates through specific steps until it converges. Initially, a tentative shape is suggested by adjusting the locations of shape points through template matching the image texture around each point. This tentative shape is then adjusted to conform to a global shape model. This cycle of adjustment continues until the model converges to an optimal shape representation that aligns well with the observed data.

In this study the ASM method is underpinned by several key mathematical theories these are Procrustes Analysis for shape alignment, Principal Component Analysis (PCA) for building a statistical shape model, and Optimization Techniques for fitting the shape model to observed data. The execution steps are described as:

- **Procrustes Analysis**

Procrustes Analysis is a method employed to align shapes by eliminating differences in translation, rotation, and scaling. The process begins with translating the shapes so that their centroids align at a common origin. Next, the shapes are scaled to ensure they are of a uniform size. Finally, the shapes are rotated to minimize the sum of squared distances between corresponding points.

Mathematically, this is achieved by considering two shapes, X and Y, each represented as a matrix of points. The Procrustes distance is minimized by finding an optimal transformation matrix, T, which effectively aligns the shapes through the optimal combination of translation, scaling, and rotation as shown in Eq. (3).

$$min_T \| X - TY \|^2 \tag{3}$$

- **Principal Component Analysis (PCA)**

Principal Component Analysis (PCA) is used to develop a statistical shape model from a set of aligned shapes. The process begins with data preparation, where each shape is represented as a vector by concatenating its coordinates. Following this, the mean shape is calculated by averaging the aligned shapes.

$$\overline{X} = \frac{1}{N} \sum_{i=1}^{N} X_i \tag{4}$$

where N is the number of shapes.

The Covariance Matrix computes the covariance matrix C of the shape vectors.

$$C = \frac{1}{N-1} \sum_{i=1}^{N} (X_i - \overline{X})(X_i - \overline{X})^T \tag{5}$$

The Eigen Decomposition perform eigen decomposition on the covariance matrix to find eigenvalues and eigenvectors.

$$Cv_i = \lambda_i v_i \tag{6}$$

Where v_i are the eigenvectors (principal components) and λ_i are the eigenvalues.

The Shape Representation represent shapes as a linear combination of the mean shape and a weighted sum of eigenvectors.

$$X \approx \overline{X} + Pb \tag{7}$$

Where P is the matrix of eigenvectors and b is the vector of weights (shape parameters).

- **Shape Model Fitting**

Fitting the shape model to new data involves finding the shape parameters b that best align the model to observed points. This process includes:

- Initial Guess: Start with an initial guess for the shape parameters.
- Iterative Optimization: Adjust the parameters to minimize the difference between the model points and the observed points.
- The optimization problem can be formulated as

$$min_b \| X_{abs} - (\overline{X} + Pb) \|^2 \tag{8}$$

where X_{abs} are the observed points.

- **ASM Mathematical Formulation**

The mathematical formulation of Active Shape Models (ASMs) involves several key steps. During the training phase, shapes are aligned using Procrustes analysis to ensure consistent orientation and scaling. Principal Component Analysis (PCA) is then applied to identify the main modes of shape variation, represented by a mean shape $\overline{X}$, a matrix of principal components P, and shape parameters b that reconstruct any shape X. Model fitting entails adjusting these parameters b based on observed landmarks X_{abs} to minimize reconstruction error. These foundational techniques allow ASMs to effectively capture and model shape variability, making them valuable for applications like facial landmark detection.

7. Proposed method

This study focuses on improving the accuracy and efficiency of facial emotion detection systems by developing a hybrid approach that integrates both full facial features and upper facial landmarks. The framework is based on a dual-input Convolutional Neural Network (CNN) model, where one input processes a full 48 × 48 facial image, and the other processes upper facial landmarks. The full facial features capture the overall emotional expression, while the landmarks provide focused cues from critical facial regions. This combination is expected to yield better emotion classification accuracy compared to traditional methods that rely solely on full facial images.

An adaptive Genetic Algorithm (GA) is employed to optimize the CNN architecture, refining the number of filters and layers for each input. The system is designed to adapt to different emotional expressions under varied conditions, using well-known datasets such as FER2013 and CK+ for training and evaluation. Performance is measured using several optimizers, including an adaptive Stochastic Gradient Descent (SGD) that enhances convergence and minimizes loss.

The methodology begins by pre-processing the dataset to extract both full facial images and upper facial landmarks. These inputs are then fed into the CNN, where feature extraction takes place through multiple convolutional layers. The extracted features from both inputs are concatenated and processed through fully connected layers with L2 regularization to avoid overfitting. An adaptive GA is used to select the optimal CNN structure, improving the balance between complexity and performance.

The study further evaluates various optimization techniques, comparing the proposed method with conventional CNN approaches. The performance is validated using metrics like accuracy, convergence speed, and loss reduction, ensuring that the hybrid method offers a significant improvement in emotion detection accuracy. Fig. 3 illustrates the process and structure of the Convolutional Neural Network.

7.1. Genetic Algorithm (GA)

Genetic Algorithms (GAs) are heuristic search algorithms inspired by natural evolution. They employ a process similar to natural selection, where the most fit individuals are chosen to produce the next generation of offspring [14]. GAs serve as effective problem-solving strategies for finding optimal solutions, especially in cases where little is initially understood about the problem. In the context of Convolutional Neural Networks (CNNs), features are generated automatically, but optimization is needed due to the inherent complexity of large-scale parallel operations and the presence of overlapping redundant features. This study employs evolutionary algorithms for feature selection, aiming to eliminate irrelevant deep features. This approach reduces computational complexity, minimizes overfitting, and enhances representation quality. The algorithm's effectiveness in selecting optimal filter numbers is evaluated across two diverse datasets to assess classification performance and variance in filter features. Fig. 4 illustrates the fundamental operations involved in the genetic algorithm process.

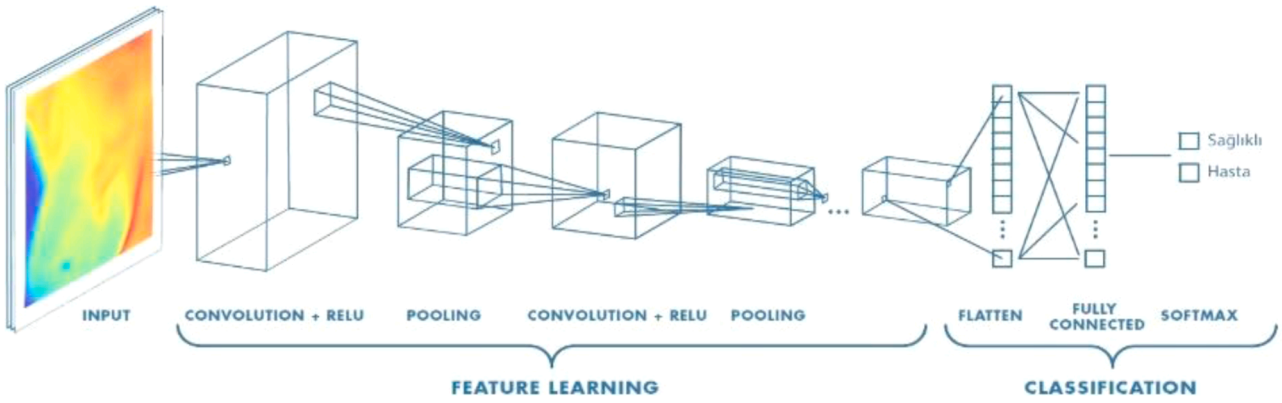

Fig. 3. Conventional CNN structure [34].

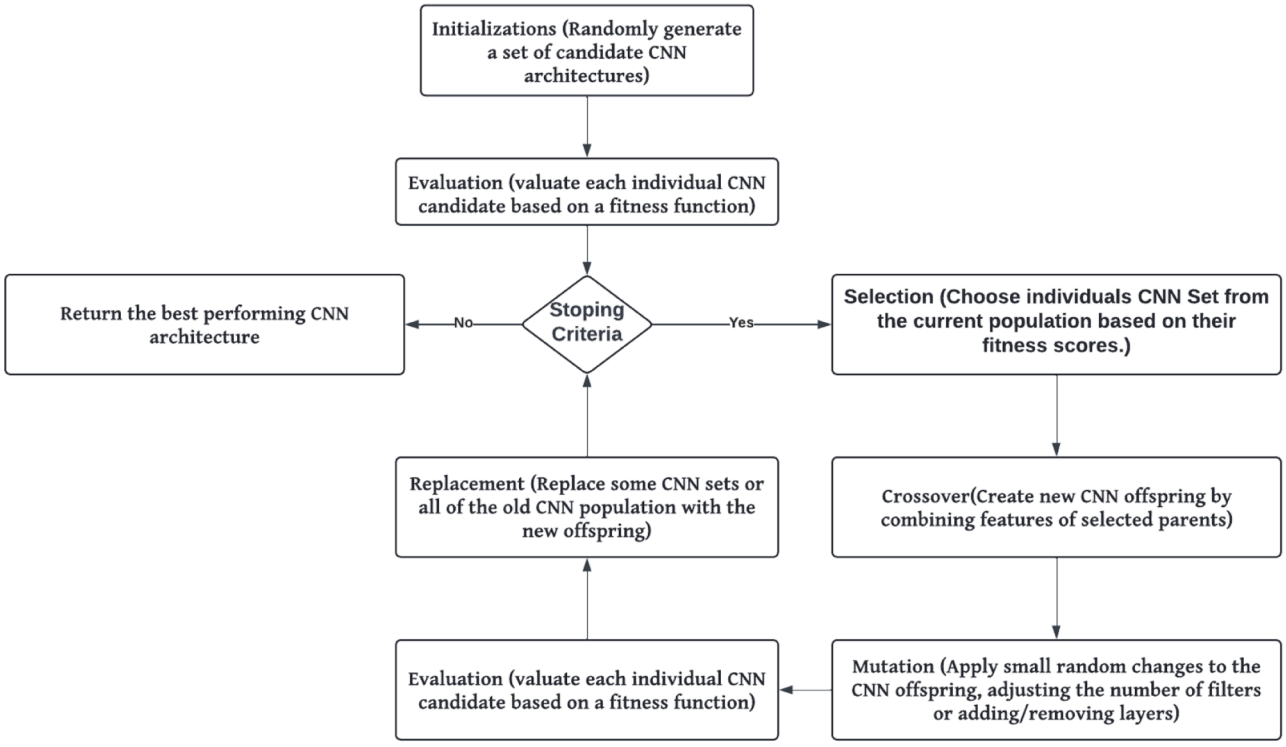

Fig. 4. GA Block Diagram.

7.2. Evolutionary Algorithm

Evolutionary computation encompasses several key stages, including gene expression, gene generation, fitness evaluation, selection, and genetic operations. In the context of Genetic Algorithms (GAs), the process begins with an initial chromosome made up of randomly generated strings. A fitness function is then employed to assess the quality of each candidate solution based on the interpretation of these strings. Selection methods are utilized to identify individuals for genetic operations, which encompass mating and mutation. During the mating phase, segments of parent strings are exchanged at random points. This iterative process continues until specific termination conditions are met.

Optimizing Convolutional Neural Networks (CNNs) presents unique challenges compared to traditional evolutionary networks, primarily due to the significant computational demands of processing multiple generations for a single entity. To address this, this study introduces a method that reduces computational overhead by separating the learning and testing phases. By leveraging GA to eliminate redundant filters across all CNN layers, the approach aims to streamline computation while enhancing overall network efficiency.

The optimization process commences with **initialization**, where an initial population of candidate CNN architectures is created. This involves randomly generating a diverse set of individuals, each defined by varying configurations of filters and layers. This diversity is essential, as it offers a wide array of potential solutions for the optimization challenge.

Following initialization, the **evaluation** step occurs. Each individual in the population is assessed using a fitness function, which quantitatively measures performance metrics such as validation accuracy or loss derived from a training dataset. This evaluation ensures that the algorithm can identify well-performing architectures that may require further refinement.

Once evaluation is complete, the **selection** phase begins. In this stage, individuals from the current population are selected to serve as parents for the next generation, based on their fitness scores. Techniques like tournament selection or roulette wheel selection are commonly used

to enhance the likelihood that higher-performing architectures will contribute to subsequent generations, guiding the population toward optimal solutions.

The next phase is **crossover**, during which new offspring are created by combining features from the selected parents. This may involve exchanging entire layers or mixing the number of filters used. The objective is to inherit successful traits from parents, potentially resulting in even better-performing architectures.

During the **mutation** phase, small random adjustments are introduced to the offspring. These modifications may include changing the number of filters or adding or removing layers. Mutation plays a critical role in maintaining diversity within the population and helps prevent premature convergence on suboptimal solutions.

After the offspring have been created, the **replacement** step takes place. In this phase, some or all of the previous population is replaced with the newly generated offspring. This replacement ensures that the population remains diverse, facilitating the exploration of a broader solution space.

The algorithm then conducts a **termination check**, evaluating whether any stopping criteria have been satisfied. Common criteria include reaching a predetermined maximum number of generations or achieving convergence in the fitness scores of the population. This step is crucial for determining when to conclude the optimization process.

Finally, once the termination conditions are met, the algorithm outputs the best architecture identified throughout the process. This optimal individual represents the most effective CNN configuration discovered, encapsulating the culmination of the evolutionary optimization efforts.

7.3. GA fitness function

For each randomly generated initial population, the CNN trained using training data, which produces a CNN architecture accuracy. The fitness function python code is given as the Evolutionary Selection of CNN filters. The necessary code to carry out the selection of the number CNN filter is given as

```
def fits(pop, X,y, epochs)
  pop_accuracy = []
  for i in range (population.shap[0]):
    nfilters = pop [i][0:3]
    sfilters = pop [i][3:]
    model = cnn_model(nfilters, sfilters)
    K = model.fit(X,y,batch_size=32, epochs=epochs)
    accuracy = K.history["accuracy"]
    pop_accuracy.append(max(accuracy))
```

The Genetic Algorithm (GA) code introduced bellow is used for optimizing Convolutional Neural Networks (CNNs). It defines the `CNNModel` class, which establishes the architecture of the CNN with customizable filter configurations. The `initialize_population` function creates an initial population of candidate architectures by randomly generating filter counts and sizes. The fitness of each model is assessed through the `evaluate_fitness` function, which trains the models on the training dataset and measures their accuracy as fitness scores. The `select_parents` function identifies the best-performing models based on these scores, while the `crossover` function generates new offspring by averaging the filter configurations of selected parents. To maintain genetic diversity, the `mutate` function introduces random changes to some offspring. Finally, the `genetic_algorithm` function orchestrates the entire process, iterating through generations to refine the CNN architectures and ultimately outputting the best model identified throughout the optimization process.

```
# Define the CNN model class
class CNNModel(Sequential):
  def __init__(self, nfilters, sfilters):
    super().__init__()
    tf.random.set_seed(0)

    for nfilter, sfilter in zip(nfilters, sfilters):
      self.add(Conv2D(nfilter, kernel_size=(sfilter, sfilter), padding="same",
activation="relu"))
      self.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

    self.add(Flatten())
    self.add(Dropout(0.3))
    self.add(Dense(512, activation="relu"))
    self.add(Dense(7, activation="softmax"))

    self.compile(optimizer="sgd", loss="categorical_crossentropy", metrics=
["accuracy"])

# Initialize population of CNN architectures
def initialize_population(pop_size, filter_ranges):
  population = []
  for _ in range(pop_size):
    nfilters = np.random.randint(filter_ranges[0][0], filter_ranges[0][1], size=3)
    sfilters = np.random.randint(filter_ranges[1][0], filter_ranges[1][1], size=3)
    population.append((nfilters, sfilters))
  return population

# Evaluate the fitness of each individual
def evaluate_fitness(population, X_train, y_train):
  fitness_scores = []
  for nfilters, sfilters in population:
    model = CNNModel(nfilters, sfilters)
    model.fit(X_train, y_train, epochs=5, verbose=0) # Adjust epochs as needed
    _, accuracy = model.evaluate(X_train, y_train, verbose=0)
    fitness_scores.append(accuracy)
  return fitness_scores

# Select parents for the next generation
def select_parents(population, fitness_scores, num_parents):
  parents = np.array(population)[np.argsort(fitness_scores)[-num_parents:]]
  return parents

# Crossover to create new offspring
def crossover(parents):
  offspring = []
  for _ in range(len(parents) // 2):
    parent1, parent2 = np.random.choice(parents, 2, replace=False)
    # Combine filters and sizes
    nfilters = (parent1[0] + parent2[0]) // 2
    sfilters = (parent1[1] + parent2[1]) // 2
    offspring.append((nfilters, sfilters))
  return offspring

# Mutate the offspring
def mutate(offspring, mutation_rate=0.1):
  for i in range(len(offspring)):
    if np.random.rand() < mutation_rate:
      nfilters, sfilters = offspring[i]
      # Randomly alter filter count or size
      if np.random.rand() < 0.5:
        nfilters = np.random.randint(1, 64, size=3)
      else:
        sfilters = np.random.randint(3, 7, size=3)
      offspring[i] = (nfilters, sfilters)
  return offspring

# Main genetic algorithm function
def genetic_algorithm(X_train, y_train, pop_size=10, generations=20):
  filter_ranges = [(1, 64), (3, 7)] # Example ranges for filters and sizes
  population = initialize_population(pop_size, filter_ranges)

  for generation in range(generations):
    fitness_scores = evaluate_fitness(population, X_train, y_train)
    print(f'Generation {generation}, Best Fitness: {max(fitness_scores)}')

    parents = select_parents(population, fitness_scores, pop_size // 2)
    offspring = crossover(parents)
    offspring = mutate(offspring)

    # Replacement: keep the best parents and add new offspring
```

(continued on next column)

(continued)

(continued on next page)

(*continued*)

```
    population = list(parents) + offspring

  # Output the best architecture
  best_idx = np.argmax(fitness_scores)
  return population[best_idx]
```

The GA recommended optimized CNN model and the implementation of the adaptive SGD is given as

```
class AGD(tf.keras.optimizers.Optimizer):
  def __init__(self, learning_rate=0.01, beta=0.9, lambda_param=0.1, name="AGD",
kwargs):
    super().__init__(name, kwargs)
    self.learning_rate = learning_rate
    self.beta = beta
    self.lambda_param = lambda_param
    self.prev_gradients = {}
    self.prev_errors = []

  def _create_slots(self, var_list):
    for var in var_list:
      self.add_slot(var, "prev_gradient")
  def _resource_apply_dense(self, grad, var, apply_state=None):
    prev_gradient = self.get_slot(var, "prev_gradient")
    if var in self.prev_gradients:
      adaptive_momentum = self.lambda_param / (1 + tf.exp(-(self.prev_errors[-1]
+ self.prev_errors[-2])))

      # Update previous gradients
      prev_gradient.assign(self.beta * prev_gradient + (1 - self.beta) * grad)
      var.assign_sub(self.learning_rate * (adaptive_momentum * prev_gradient))
    else:
      # First update
      prev_gradient.assign(grad)
      var.assign_sub(self.learning_rate * grad)
  def apply_gradients(self, grads_and_vars, name=None,
experimental_aggregate_gradients=True):
    # Store error for adaptive momentum calculation
    self.prev_errors.append(self.compute_loss(grads_and_vars))
    super().apply_gradients(grads_and_vars, name,
experimental_aggregate_gradients)

  def compute_loss(self, grads_and_vars):
    # Compute a simple loss metric (could be modified)
    loss = sum(tf.reduce_mean(tf.square(grad)) for grad, _ in grads_and_vars)
    return loss

class CNNModel(Sequential):
  def __init__(self, nfilters, sfilters):
    super().__init__()

    tf.random.set_seed(0)
    # First convolutional layer
    self.add(Conv2D(nfilters[0], kernel_size=(sfilters[0], sfilters[0]),
padding="same", activation="relu"))
    self.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

    # Second convolutional layer
    self.add(Conv2D(nfilters[1], kernel_size=(sfilters[1], sfilters[1]),
padding="same", activation="relu"))
    self.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

    # Third convolutional layer
    self.add(Conv2D(nfilters[2], kernel_size=(sfilters[2], sfilters[2]),
padding="same", activation="relu"))
    self.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))

    # Flatten the output from the convolutional layers
    self.add(Flatten())
    self.add(Dropout(0.3))
    self.add(Dense(512, activation="relu"))
    self.add(Dense(7, activation="softmax"))

    # Compile the model with the AGD optimizer
    self.compile(optimizer=AGD(),
          loss="categorical_crossentropy",
          metrics=["accuracy"])
```

The code presented above defines a class called `CNNModel` that inherits from TensorFlow's `Sequential` class, facilitating the construction of a Convolutional Neural Network (CNN) optimized through a Genetic Algorithm (GA). This specific architecture represents the outcome of the GA's evolutionary process, where candidate CNN structures were evaluated and refined based on performance metrics. The constructor (`__init__` method) accepts two parameters: `nfilters` and `sfilters`, which specify the number of filters and their sizes for each convolutional layer, respectively. To ensure reproducibility of results, a random seed is set at the beginning of the method.

The model consists of three convolutional layers, each followed by a max pooling layer. The `Conv2D` layers apply convolution operations with the specified number of filters and kernel sizes, using "same" padding to maintain the input size and the ReLU activation function to introduce non-linearity. Each convolutional layer is followed by a `MaxPooling2D` layer that downsamples the feature maps, effectively reducing their spatial dimensions.

After the convolutional and pooling layers, the output is flattened into a one-dimensional array using the `Flatten()` layer, preparing it for the fully connected layers. A dropout layer is included to mitigate overfitting by randomly setting a portion of the input units to zero during training.

The architecture concludes with two dense layers: the first fully connected layer has 512 units with ReLU activation, while the second is the output layer with 7 units and a softmax activation function, suitable for multi-class classification tasks. Finally, the model is compiled with the stochastic gradient descent (SGD) optimizer and the categorical cross-entropy loss function, using accuracy as a metric to evaluate performance. This structured approach allows for effective feature extraction and classification in various applications, demonstrating the potential of GA in optimizing CNN architectures for improved performance.

7.4. ASM implementations

In this study it is it is intended to implement the Active Shape Models (ASM) for facial landmark detection as it was introduced in Section 5 above. The implementation includes the necessary code for Procrustes analysis, PCA, and fitting the model to observed data. This code the foundational implementation related to Active Shape Models (ASM). The alignment of shapes through Procrustes analysis is a crucial step in ASM, ensuring consistent comparisons by aligning shapes to a common mean. The PCA function captures the principal modes of variation among the aligned shapes, reflecting a core idea behind ASM. Additionally, the ability to reconstruct shapes and fit the model to observed data illustrates how ASM adapts statistical models to specific shape instances. The code implements a series of functions for aligning shapes using Procrustes analysis, performing Principal Component Analysis (PCA), and fitting a shape model to observed data.

The `procrustes` function is designed to align two sets of shapes, X and Y. It first computes the mean of each shape and centers the data by subtracting these means. The centered shapes are then normalized to have unit norm, allowing for comparisons that are invariant to scale. Singular Value Decomposition (SVD) is applied to the product of the scaled and centered shapes to compute the rotation matrix R, which minimizes the distance between the two shapes in a least-squares sense. The function returns the aligned version of X, the rotation matrix R, the means of X and Y, and the norms of X and Y.

The code then aligns a set of shapes (stored in the variable `shapes`) to a mean shape using the `procrustes` function. Initially, the first shape is taken as the mean shape. The code iterates through the first ten shapes, aligning each one to the current mean shape and updating the mean shape to the newly aligned shape. This iterative process ensures that all shapes are aligned to a common reference, improving their comparability.

The `pca` function performs PCA on the aligned shapes. It first

flattens the shapes into a 2D array, centering the data by subtracting the mean. SVD is applied to this centered data to extract eigenvectors and eigenvalues, representing the principal components and their corresponding variances. These components capture the most significant modes of variation in the shapes. The function returns the eigenvectors, eigenvalues, and the mean shape.

The `reconstruct_shape` function reconstructs a shape from the mean shape and a set of PCA coefficients (parameters) *b*. It adds the contribution of the eigenvectors scaled by these coefficients to the mean shape, yielding a new shape representation. The output is reshaped into the original two-dimensional format.

The `fit_shape_model` function fits the shape model to new observed data. It takes an observed shape, the mean shape, and the eigenvectors as inputs. The observed shape is flattened, and the PCA coefficients *b* are computed by projecting the difference between the observed shape and the mean shape onto the eigenvectors. The fitted shape is then reconstructed using the mean shape and the PCA coefficients, resulting in a shape that best represents the observed data while adhering to the learned shape model.

Finally, the code simulates new observed data by perturbing the mean shape with random noise. This perturbed shape is then fitted to the shape model using the `fit_shape_model` function, producing a fitted shape that incorporates the characteristics of the observed data while conforming to the learned model. A snap of the code is given below:

```
def procrustes(X, Y):
    X_mean = X.mean(axis=0)
    Y_mean = Y.mean(axis=0)
    X_centered = X - X_mean
    Y_centered = Y - Y_mean
    X_norm = np.linalg.norm(X_centered)
    Y_norm = np.linalg.norm(Y_centered)
    X_scaled = X_centered / X_norm
    Y_scaled = Y_centered / Y_norm
    U, S, Vt = np.linalg.svd(X_scaled.T @ Y_scaled)
    R=U @ Vt
    X_aligned = X_scaled @ R
    return X_aligned, R, X_mean, Y_mean, X_norm, Y_norm

# Align shapes using Procrustes analysis
aligned_shapes = shapes.copy()
mean_shape = shapes[0]

for i in range(10):
    aligned_shapes[i], _, _, _, _, _ = procrustes(mean_shape, shapes[i])
    mean_shape = aligned_shapes[i]


# Function to perform PCA
def pca(X):
    X_flat = X.reshape(X.shape[0], -1)
    X_mean = X_flat.mean(axis=0)
    X_centered = X_flat - X_mean
    U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)
    eig_vectors = Vt.T
    eig_values = S2 / (X_flat.shape[0] - 1)
    return eig_vectors, eig_values, X_mean

# Perform PCA on aligned shapes
eig_vectors, eig_values, mean_shape_flat = pca(aligned_shapes)

# Function to reconstruct shape from parameters
def reconstruct_shape(mean_shape, eig_vectors, b):
    shape = mean_shape + eig_vectors @ b
    return shape.reshape(-1, 2)

# Function to fit the shape model to new observed data
def fit_shape_model(observed_shape, mean_shape, eig_vectors):
    observed_shape_flat = observed_shape.flatten()
    b = eig_vectors.T @ (observed_shape_flat - mean_shape)
    fitted_shape_flat = mean_shape + eig_vectors @ b
    fitted_shape = fitted_shape_flat.reshape(-1, 2)
    return fitted_shape, b
# Simulate new observed data (perturbed mean shape)
```

(continued on next column)

(continued)

```
observed_shape = mean_shape + np.random.randn(num_landmarks, 2) * 5

# Fit the shape model to the observed data
fitted_shape, b = fit_shape_model(observed_shape, mean_shape_flat, eig_vectors)
```

7.5. Optimizer

The optimizer proposed for this study is the Adaptive Gradient Descent (AGD) optimizer. While conventional gradient descent is commonly applied in linear regression, classification, and neural networks, it operates as a first-order optimization algorithm. This method relies on the first-order derivative of the loss function. By utilizing Backpropagation, which transfers the loss through the network layers, model parameters (or 'weights') are adjusted based on these losses to minimize overall loss.

7.5.1. Overview of BP algorithm

The traditional BP algorithm was introduced by D. E. Rumelhart etal [7], in this case, the initial weight W0, iterative increment formula is given as

$$\Delta w(n+1) = \eta\delta_o \mathrm{y}(n) + \alpha\Delta w(n)\ n = 0, 1, \ldots \tag{9}$$

the learning rate $\eta > 0$ dictates the step size in the direction opposite to the gradient of the loss function. However, this algorithm's convergence can be slow, particularly in networks with multiple hidden layers, exacerbated by the behavior of activation functions. Another challenge lies in selecting an appropriate fixed learning rate η that balances rapid learning with stability [7,22,26,29]. To address these issues, a solution proposes replacing the fixed learning rate with an adaptive function that adjusts based on changes in the loss, offering a more suitable and stable learning approach.

7.5.2. Training in back-propagation algorithm

The core principle of Backpropagation is to minimize the overall output error throughout the learning process. This error is defined as the difference between the actual output and the expected output, serving as the network's error signal. BP involves propagating this error signal from the output layer back to the input layer. The BP process [39] comprises two main stages: forward propagation and backward propagation.

During forward propagation, the BP architecture is characterized by inputs x fed into a neural network with i neurons. The weights w_{ij} represent the connections between inputs and j neurons within the hidden layer,

$$H_j = b_{in} + \sum_{i=1}^{N} \mathbf{x}_i \mathbf{w}_{ij} \tag{10}$$

Where b_i denotes the bias of the input layer, the hidden layer output passes through an activation function (f\). In this study, the softmax function is employed. The softmax activation function is mathematically represented as

$$P(y=j|x) = \frac{e^{xw_j}}{\sum_{i=1}^{K} e^{xw_j}} \tag{11}$$

After computing the overall output by multiplying the outputs of the hidden layer neurons with the weights $\mathbf{w}_{jk}$, the results are then passed through a sigmoid function, often referred to as a threshold function. This process can be illustrated as follows:

After calculating the overall output by multiplying the output of the hidden layer neurons with the hidden layer weights $\mathbf{w}_{jk}$, the results, then, pass through a sigmoid function (called threshold) as shown below;

$$y_k = b_n + \sum_{j=1}^{m} \mathbf{w}_{jk} P(y) \tag{12}$$

Where b_n is the bias of the hidden layer and k output neurons.

The network error equation E for each pattern P is calculated by subtracting the overall output o from the target t,

$$E = \frac{1}{2} \sum_{j=1}^{p} (t_j - o_j)^2 \tag{13}$$

The weight update equation, incorporating both learning and momentum terms, is given as

$$\Delta \mathbf{w}_{ji}(n+1) = \eta \delta_y \mathbf{x}_i(n) + \alpha \Delta \mathbf{w}_{ji}(n) \tag{14}$$

$$\Delta \mathbf{w}_{kj}(n+1) = \eta \delta_o y_i(n) + \alpha \Delta \mathbf{w}_{kj}(n) \; n = 0, 1, \ldots \tag{15}$$

The bias update equation is given as

$$b_j(n) = b_j(n) + \eta f(H_j)\left(1 - f(H_j)\right) \mathbf{x}_i(n) \mathbf{w}_j(n) e(n) \tag{16}$$

7.6. Adaptive momentum term

In this study, the adaptive momentum equation [40] is represented as follows:

$$\alpha(n) = \frac{\lambda}{1 + \left(e^{-(error(t) + error(t-1))}\right)} \tag{17}$$

Where $\lambda < \frac{2-2\beta}{max\ eigen\ value\ of \mathbf{R}_{xx}}$ and the β is the forgetting factor $(0 \ll \beta < 1)$,

The initial values of β are sufficiently large, causing the term to be close to unity. Consequently, the initial $\alpha(n)$ will be relatively large. This allows us to rewrite Eqs. (14) and (15) as

$$\Delta \mathbf{w}_{ji}(n+1) = \eta \delta_y \mathbf{x}_i(n) + \left(\frac{\lambda}{1 + \left(e^{-(E(t) + E(t-1))}\right)}\right) \Delta \mathbf{w}_{ji}(n) \tag{18}$$

$$\Delta \mathbf{w}_{kj}(n+1) = \eta \delta_o y_i(n) + \left(\frac{\lambda}{1 + \left(e^{-(E(t) + E(t-1))}\right)}\right) \Delta \mathbf{w}_{kj}(n), \; n = 0, 1, \ldots \tag{19}$$

Where n represents the number of iterations, and $\Delta \mathbf{w}$ is defined as updating the weights.

This adaptive momentum term is used with a standard SGD optimizer to formulate a customized Adaptive SGD optimizer. The modification to the standard Stochastic Gradient Descent (SGD) that helps to accelerate convergence, especially in the face of noisy gradients or when the gradients are small. The momentum term incorporates information from the previous gradients to smooth out the updates.

7.7. Hybrid facial emotion feature extraction method

In this study, the goal is to enhance the accuracy of emotion detection by combining upper facial landmark features with full facial features processed by a convolutional neural network (CNN). The process begins by selecting an optimal CNN structure using an adaptive Genetic Algorithm (GA), which optimizes the number of CNN layers and filters based on a small portion of the dataset. After optimization, the dataset is split into training and testing sections. For each image, the upper half (top 24 rows of a 48 × 48 pixel image) is extracted and flattened into a one-dimensional array of 24 features to represent the upper facial region. A dual-input neural network architecture is then designed, with one input for the full 48 × 48 facial image processed through a CNN, and a second input for the upper facial features. These inputs are concatenated and fed into dense layers with L2 regularization. The model is trained and evaluated using various optimizers to identify the best-performing configuration, with the best performance metrics plotted and compared. This methodology allows the model to utilize detailed pixel information alongside focused landmark features, improving its ability to detect emotions accurately.

To compare the proposed method with a conventional CNN for facial emotion detection, a genetically optimized CNN model is designed and used with the same dataset (FER2013) to detect the facial emotions. The results of the conventional method and the optimized hybrid method are compared, and several statistical analyses are conducted on the results to verify long-term accuracy and stability.

8. Contribution

This study presents several main contributions to the field of facial emotion recognition and CNN optimization:

1. Enhanced Model for Emotion Classification: We propose a novel model that combines upper facial landmark features extracted through Active Shape Models (ASM) with comprehensive facial features processed by Convolutional Neural Networks (CNN). This dual-feature integration improves the model's ability to classify facial emotions accurately.
2. Advanced Optimizer Implementation: By employing a fully adaptive Stochastic Gradient Descent (SGD) optimizer, we enhance the model's convergence speed and efficiency. Our approach showcases the advantages of adaptive learning rates in achieving optimal performance during training.
3. Application of Genetic Algorithms: The study introduces an adaptive Genetic Algorithm (GA) to optimize the structure of the CNN, demonstrating an effective method for refining model architecture. This innovative integration allows for the systematic evolution of CNN configurations tailored to specific tasks.
4. Robust Validation Across Datasets: The model was rigorously tested on two distinct datasets, providing strong empirical evidence of its effectiveness. By comparing multiple optimizers, we validate the superiority of the adaptive SGD optimizer, which achieved an impressive accuracy of 96.87 %.
5. Consistent Performance Metrics: Our validation tests, including cross-validation and statistical analysis, confirmed that the model with the adaptive SGD optimizer consistently delivered high performance, achieving a mean accuracy of 96.376 % with a low standard deviation of 0.8. These results highlight the model's robustness and reliability across different scenarios.

Finally, this work advances the understanding of emotion recognition through innovative model design and optimization strategies, contributing valuable insights for researchers and practitioners in the field.

9. Results and analysis

In this study, we employed a K-fold cross-validation approach to enhance the reliability of our validation accuracy results. Specifically, the dataset was divided into K subsets, or folds. For our analysis, we selected $K = 5$, meaning the dataset was split into five equal parts. The model was trained and validated five times, with each fold serving as the validation set once while the remaining four folds were utilized for training. This iterative process allowed us to obtain a robust estimate of the model's performance, as the final validation results were averaged across all five iterations. This approach ensures that every data point is used for both training and validation, minimizing the risk of overfitting and providing a more generalizable evaluation of the model's effectiveness. The results of both the conventional and proposed methods are presented below in Table 1 and Table 2. The tables contain the validation accuracy of both conventional and proposed method. For the testing purpose each method is tested with five different optimizers for landmark optimizers plus a customized fully adaptive SGD optimizer (ASGD). The validation accuracy obtained during the tests are plotted at

Table 1
Conventional CNN model Validation Accuracy.

Iteration	Val_Acc_Adam	Val_Acc_RMSprop	Val_Acc_Adagard	Val_Acc_Adadelta	Val_Acc_ASG
1					
10	34.32	32.68	23.95	44.45	44.45
20	58.91	49.88	47.97	61.78	64.77
30	65.17	70.67	62.58	74.09	77.69
40	72.82	77.96	70.16	81.23	87.39
50	78.19	85.91	70.10	84.56	89.91
60	82.41	9142	71.64	89.13	92.04
70	82.74	91.89	71.89	89.05	92.05
80	82.54	91.89	71.38	89.06	92.05
90	82.65	91.25	71.37	89.04	92.05
100	82.66	91.89	71.41	89.05	92.06

Table 2
Proposed method validation accuracy using Fer2023 dataset.

Iteration	Val_Acc_Adam	Val_Acc_RMSprop	Val_Acc_Adagard	Val_Acc_Adadelta	Val_Acc_ASGD
1	16.45	11.90	18.98	13.92	21.19
10	46.22	65.74	41.77	63.45	85.09
20	73.26	87.30	70.88	77.22	95.03
30	86.17	93.89	83.48	92.22	96.24
40	88.38	94.04	83.93	92.23	95.42
50	87.58	94.21	83.93	92.03	96.44
60	87.68	94.46	83.93	92.05	96.40
70	88.38	94.03	83.93	92.74	96.87
80	87.58	93.35	83.93	92.05	96.87
90	87.68	94.24	83.93	92.65	96.87
100	87.60	94.45	83.93	92.45	96.87

the end of the testing process as shown in both Fig. 5 and Fig. 6. Table 1 shows the accuracy values for the CNN model for facial emotion classification with a variation of optimizers including Adam. RMSprop, Adagard, Adadelta, Adaptive SGD.

The two models with the top validation accuracy obtained against the FER2013 dataset are trained and tested against the second dataset CK+. The validation accuracy result obtained in this second test are and presented in Table 4.

Table 2 shows the validation accuracy for the hybrid top facial landmarks and the full facial CNN model with the same variation of optimizers used in first test.

Table 3: Proposed method validation accuracy using CK+ dataset for the model utilizing both RMSprop and ASGD optimizer

The results in Table 3 validates the accuracy obtained by the model against the FER2013 dataset even though there is a definite size difference between the two dataset as the CK+ dataset is much smaller than the FER2013 dataset. Fig. 5, illustrates the results of table 3.

From Fig. 7, it can be seen that the model performed in the same manner as it did when trained and tested against the FER2013 dataset which a much bigger dataset.

9.1. *Validation accuracy analysis*

The validation accuracy of the proposed method was analyzed across 100 iterations using five different optimization algorithms: Adam, RMSprop, Adagrad, Adadelta, and Adaptive_SGD (ASGD). The results from Table 2 can be summarized as, initially, ASGD started with the highest validation accuracy at 21.19 %, while Adagrad and Adam also began relatively high at 18.98 % and 16.45 %, respectively. In contrast, RMSprop had the lowest initial accuracy at 11.90 %, closely followed by Adadelta at 13.92 %. Moving to the mid-term performance, by iteration 10, ASGD significantly outperformed other optimizers, achieving 85.09 %. RMSprop and Adadelta also showed strong performance at 65.74 % and 63.45 %, respectively. Adam and Adagrad lagged behind during

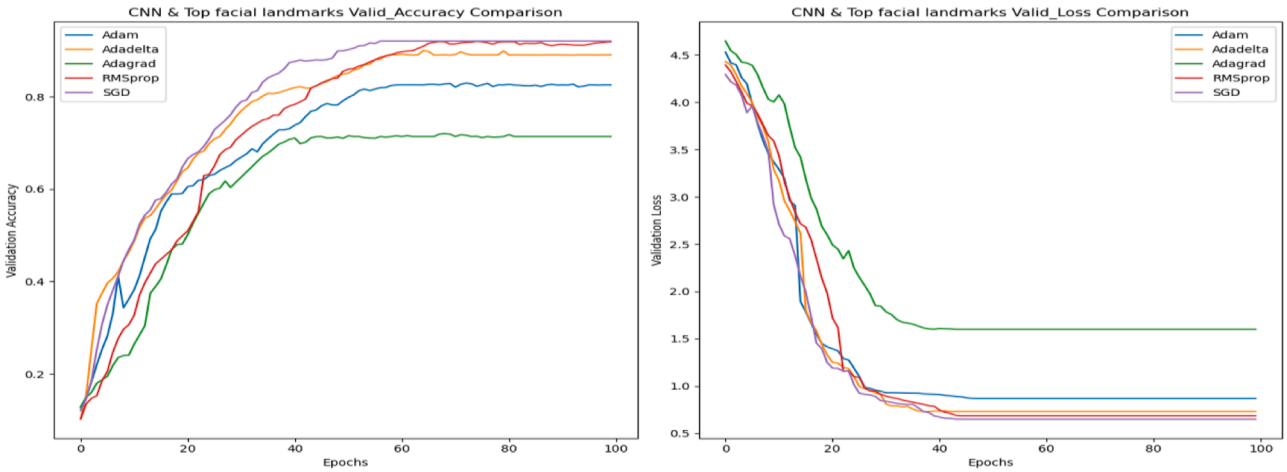

Fig. 5. CNN model Validation accuracy and validation loss for a varied number of optimizers.

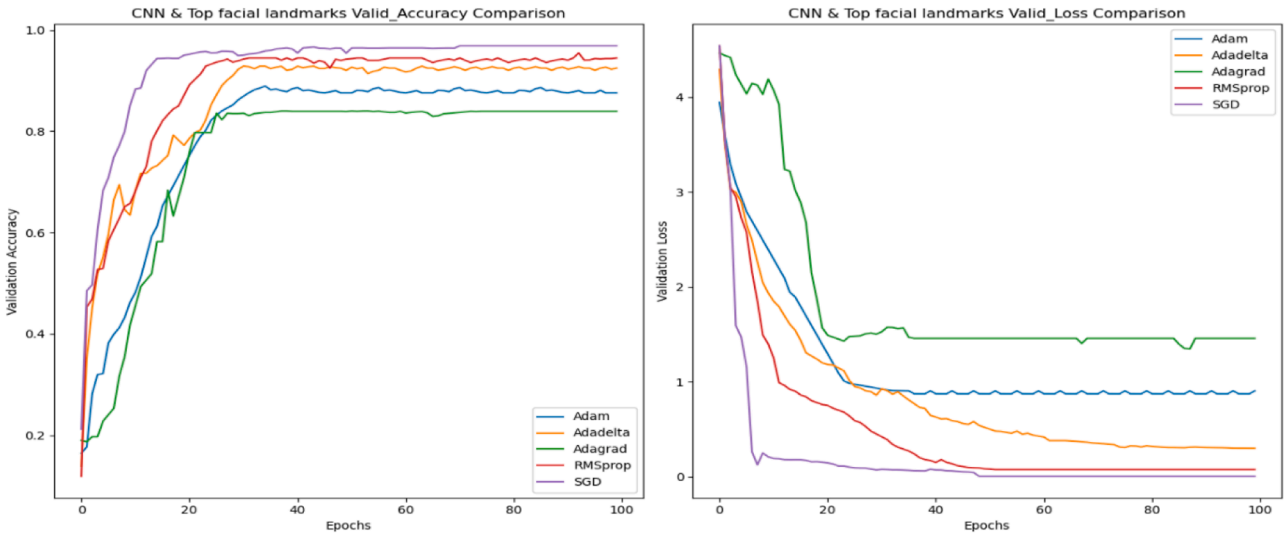

Fig. 6. Proposed method validation accuracy and validation loss for a varied number of optimizers.

Table 3
Proposed method validation accuracy using C.

Iteration	Val_Acc_Adam	Val_Acc_RMSprop	Val_Acc_Adagard	Val_Acc_Adadelta	Val_Acc_ASGD
1	13.72	10.95	11.38	10.36	20.75
10	34.46	62.57	22.53	34. 58	84.69
20	64.98	84.98	50.56	49.76	94.89
30	76.83	92.88	63.54	73.51	95.94
40	81.52	93.95	67.43	84.39	95.75
50	84.26	94.33	72.88	86.76	96.21
60	85.33	94.18	80.08	89.65	96.54
70	86.59	94.30	83.42	91.42	96.75
80	87.44	93.57	83.57	91.87	96.82
90	86.98	94.32	84.09	92.06	96.84
100	87.07	94.38	83.99	91.89	96.85

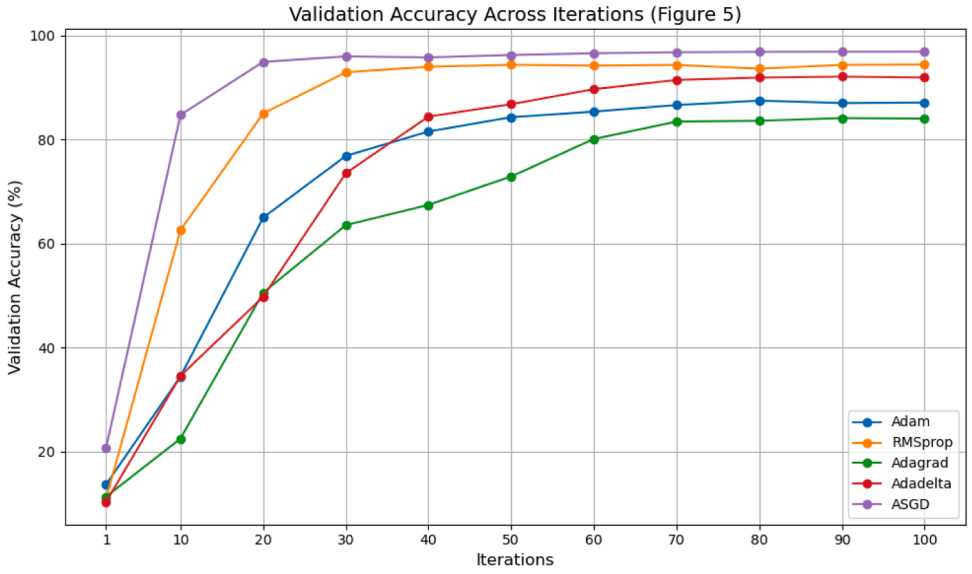

Fig. 7. Model Validation accuracies against CK+ dataset.

early to mid-iterations but showed considerable improvement over time. By the end of iteration 50, RMSprop (94.21 %) and ASGD (96.44 %) maintained superior performance, while Adadelta and Adam achieved slightly lower but still competitive results.

In the long-term performance, ASGD consistently maintained the highest validation accuracy, peaking at 96.87 %. RMSprop also performed very well, stabilizing around 94.45 % by iteration 100. Adam and Adadelta showed high but slightly lower performance than RMSprop and ASGD, with accuracies around 87.60 % and 92.45 %, respectively. Adagrad stabilized at around 83.93 %, indicating it was

less effective compared to the other optimizers over the long term.

The results indicate that the model utilizing ASGD consistently shows the highest validation accuracy across all iterations, making it the most effective optimizer for this particular task. RMSprop also demonstrates strong performance, especially from mid to late iterations. Adam and Adadelta perform well but lag slightly behind ASGD and RMSprop, particularly in early iterations. The model utilizing the Adagrad optimizer shows the least improvement over iterations, stabilizing at a lower accuracy compared to the other optimizers. These results suggest that for this specific task, ASGD is the preferred optimizer, with RMSprop as a strong alternative. Adam and Adadelta are also viable options but may require more tuning or be better suited for different types of tasks. Adagrad appears to be the least effective among the five optimizers tested.

9.2. Results accuracy validation

To ensure that the validation accuracy results are reliable and generalizable to other similar cases, in this study several steps were taken:

- Cross-Validation: in this case, the dataset is split into K subsets (folds). The model is then trained and validated 25 times, each time using a different fold as the validation set and the remaining folds as the training set. The validation results are averaged to get a more reliable estimate of the model's performance.
- External Validation: In this case, the models were tested on different but similar datasets to check their generalizability. This helps confirm that the observed performance is not specific to the original dataset. For this study, two datasets are used to satisfy this validation test, FER2012 and CK+ are used to complete this test.
- Statistical Significance Testing: In this study a number of statistical tests are carried out to verify the accuracy and validity of the proposed method. The tests included Paired *t*-tests: Used to compare the performance of different models and optimizers, determining if the differences in performance are statistically significant. The Wilcoxon Signed-Rank Test is carried out, the test is a non-parametric alternative to the paired *t*-test, used when the data does not follow a normal distribution. Then the ANOVA (Analysis of Variance) test is carried out this test is used to compare the means of three or more groups. Significant differences were followed up with post-hoc tests like Tukey's HSD to identify specific group differences. The final statistical test to be carried out is the Friedman Test: A non-parametric test for comparing more than two related groups, useful for multiple models tested on the same datasets.

9.3. Statistical tests for comparing model performances

From the results in Table 2, it can be concluded that the algorithm utilizing ASGD achieved the highest mean accuracy (96.376 %) with the smallest standard deviation (0.800), indicating consistent high performance. The algorithm using RMSprop also obtained a high mean accuracy (92.782 %) but had a higher standard deviation (3.072), suggesting more variability. The algorithms utilizing Adadelta and Adam showed moderate mean accuracy with larger standard deviations, indicating more fluctuation in performance. The algorithm using Adagrad had the lowest mean accuracy (81.230 %) and a relatively high standard deviation (5.789), suggesting it is less effective and more variable. Statistical tests yielded the following results:

- Paired t-Test: Adam vs. RMSprop: t-stat = −5.422, p-value = 0.0056, indicating a statistically significant difference in performance between Adam and RMSprop.
- Wilcoxon Signed-Rank Test: Adam vs. RMSprop: w-stat = 0.0, p-value = 0.0625, indicating the difference between Adam and RMSprop is not statistically significant at the 5 % significance level using this non-parametric test.
- ANOVA (Analysis of Variance): F-stat = 7.061, p-value = 0.0010, indicating significant differences in the performance of at least one pair of optimizers.
- Friedman Test: Chi-square = 20.0, p-value = 0.0005, suggesting significant differences in performance across the optimizers.

In this study, the F-test is used to compare the variances of two populations to determine if they are significantly different. Table 4 presents the F-test results and their interpretations.

The F-test results indicate that ASGD has significantly different performance variance compared to Adam, RMSprop, and Adadelta, suggesting it behaves differently in terms of stability and consistency. There are no significant differences in variances between the other pairs of optimizers, implying they have similar stability and consistency in their performance.

10. Conclusions

The study proposed an enhanced model for classifying facial emotions in images. This model integrates upper facial landmarks features extracted using Active Shape Models (ASM) with full facial features processed using Convolutional Neural Networks (CNN). The combined features are then fed into a fully dense layer with L2 regularization. Additionally, a fully adaptive Stochastic Gradient Descent (SGD) based optimizer was employed to improve the model's convergence, and an adaptive Genetic Algorithm (GA) was used to optimize the CNN structure.

The model was trained and tested on two different datasets to validate its effectiveness. To verify the superiority of the fully adaptive optimizer in enhancing the model's convergence, multiple optimizers were used during the training and testing process. The results showed that the proposed model, utilizing the adaptive SGD optimizer, significantly outperformed other optimizer combinations, achieving an accuracy of 96.87 %.%, as shown in Table 2. The results shown in Table 3, confirms the validation accuracy obtained by the proposed method as the results shows that the model consistent performance against a much smaller dataset. Which indicates that the proposed method can achieve similar performance against datasets of varying size.

Various validation tests, including cross-validation, external validation, and statistical validation, confirmed that the model with the adaptive SGD optimizer achieved the highest mean accuracy of 96.376 % with a standard deviation of 0.8, indicating consistently high performance. An F-test comparing the performance variance demonstrated that the model with the adaptive SGD optimizer had significantly different performance variance.

The optimization method adopted in this study can be easily integrated into future CNN model structures and applications, potentially having a considerable impact on improving model accuracy. Future tasks include addressing real-time problems under challenging conditions, such as low light, varied facial positions and angles, and an

Table 4
F-test results.

Comparison	F-stat	p-value	Significant difference in variance
Adam vs. RMSprop	4.3402	0.1041	No significant
Adam vs. Adagrad	1.2216	0.8035	No significant
Adam vs. Adadelta	0.8756	0.8829	No significant
Adam vs. ASGD	63.9504	0.0006	Significant
RMSprop vs. Adagrad	0.2815	0.1762	No significant
RMSprop vs. Adadelta	0.2017	0.0861	No significant
RMSprop vs. ASGD	14.7364	0.0216	Significant
Adagrad vs. Adadelta	0.7165	0.5090	No significant
Adagrad vs. ASGD	4.4751	0.0913	No significant
Adadelta vs. ASGD	6.2487	0.0381	Significant

increasing number of facial landmarks.

Author contributions

The following represents the substantial contributions of individual authors: Conceptualization, NA, AO, FA; methodology, NA, AO. FT, FA; performing the experiments, NA, MT, FT; analyzing the data, NA, AO; writing the manuscript, NA, FA, MT; providing scientific supervision of manuscript, NA, FA. All authors have read and agreed to the published version of the manuscript.

Funding

Not applicable.

Ethical declarations

We confirm that all methods were carried out in accordance with relevant guidelines and regulations.

Informed consent statement

Not applicable.

Data availability

Not applicable, the datasets used in this research are publicly available. All the datasets used in this paper are referenced directly via a listing in the references.

CRediT authorship contribution statement

Naim Ajlouni: Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Adem Özyavaş:** Writing – original draft, Software, Investigation, Formal analysis, Conceptualization. **Firas Ajlouni:** Methodology, Conceptualization. **Faruk Takaoğlu:** Software, Formal analysis. **Mustafa Takaoğlu:** Writing – original draft, Methodology, Data curation.

Conflict of Interest

The authors declare no conflict of interest.

References

[1] S.M. Alarcão, M.J. Fonseca, Emotions recognition using EEG signals: a survey, IEEE Trans. Affect. Comput. 10 (3) (1 July-Sept. 2019) 374–393, https://doi.org/10.1109/TAFFC.2017.2714671.
[2] S. Li, W. Deng, Blended emotion in-the-wild: multi-label facial expression recognition using crowdsourced annotations and deep locality feature learning, Int. J. Comput. Vis. 127 (6–7) (2019) 884–906, https://doi.org/10.1007/s11263-018-1131-1.
[3] B. Martinez, M.F. Valstar, B. Jiang, M. Pantic, Automatic analysis of facial actions: a survey, IEEE Trans. Affect. Comput. 10 (3) (1 July-Sept. 2019) 325–347, https://doi.org/10.1109/TAFFC.2017.2731763.
[4] S. Li, W. Deng, Blended Emotion in-the-Wild: multi-label Facial Expression Recognition Using Crowdsourced Annotations and Deep Locality Feature Learning, Int. J. Comput. Vis. 127 (6–7) (2019) 884–906, https://doi.org/10.1007/s11263-018-1131-1.
[5] G. Zhao, X. Li, Automatic Micro-Expression Analysis: open Challenges, Front. Psychol. 10 (2019) 1833, https://doi.org/10.3389/fpsyg.2019.01833.
[6] Y. Huang, F. Chen, S. Lv, X. Wang, Facial Expression Recognition: a Survey, Symmetry. (Basel) 11 (10) (2019) 1189, https://doi.org/10.3390/sym11101189.
[7] R.G. Harper, A.N. Wiens, J.D. Matarazzo, Nonverbal Communication: The State of the Art, 1st Editio, Wiley, New York, NY, USA, 1978.
[8] Y.-H. Byeon, K.-C Kwak*, Facial expression recognition using 3D convolutional neural network, Int. J. Adv. Comput. Sci. Appl. 5 (12) (2014), https://doi.org/10.14569/IJACSA.2014.051215.
[9] P. Liu, S. Han, Z. Meng, Y. Tong, Facial Expression Recognition via a Boosted Deep Belief Network, in: 2014 IEEE Conference on Computer Vision and Pattern Recognition, IEEE, 2014, pp. 1805–1812, https://doi.org/10.1109/CVPR.2014.233.
[10] C. Shan, S. Gong, P.W. McOwan, Facial expression recognition based on Local Binary Patterns: a comprehensive study, Image Vis. Comput. 27 (6) (2009) 803–816, https://doi.org/10.1016/j.imavis.2008.08.005.
[11] C.-R. Chen, W.-S. Wong, C.-T. Chiu, A 0.64 mm^{2} Real-Time Cascade Face Detection Design Based on Reduced Two-Field Extraction, IEEE Trans. Very. Large Scale Integr. VLSI. Syst. 19 (11) (2011) 1937–1948, https://doi.org/10.1109/TVLSI.2010.2069575.
[12] D. Lakshmi, R. Ponnusamy, Facial emotion recognition using modified HOG and LBP features with deep stacked autoencoders, Microprocess. Microsyst. 82 (2021) 103834, https://doi.org/10.1016/j.micpro.2021.103834.
[13] D.D. Agrawal, S.R. Dubey, A.S. Jalal, Emotion recognition from facial expressions based on multi-level classification, Int. J. Comput. Vis. Robot. 4 (4) (2014) 365, https://doi.org/10.1504/IJCVR.2014.065571.
[14] M.F. Valstar, M. Mehu, Bihan Jiang, M. Pantic, K. Scherer, Meta-analysis of the first facial expression recognition challenge, IEEE Trans. Syst. Man Cybern. Part B (Cybernetics) 42 (4) (2012) 966–979, https://doi.org/10.1109/TSMCB.2012.2200675.
[15] F. Abdat, C. Maaoui, A. Pruski, Human-computer interaction using emotion recognition from facial expression, in: 2011 UKSim 5th European Symposium on Computer Modeling and Simulation, IEEE, 2011, pp. 196–201, https://doi.org/10.1109/EMS.2011.20.
[16] G. Littlewort, J. Whitehill, T.-F. Wu, N. Butko, P. Ruvolo, J. Movellan, M. Bartlett, The motion in emotion – A CERT based approach to the FERA emotion challenge, in: Face and Gesture 2011, IEEE, 2011, pp. 897–902, https://doi.org/10.1109/FG.2011.5771370.
[17] D. Ghimire, S. Jeong, J. Lee, S.H. Park, Facial expression recognition based on local region specific features and support vector machines, Multimed. Tools. Appl. 76 (6) (2017) 7803–7821, https://doi.org/10.1007/s11042-016-3418-y.
[18] Tang, C., Xu, P., Luo, Z., Zhao, G., & Zou, T. (2015). *Automatic facial expression analysis of students in teaching environments*. https://doi.org/10.1007/978-3-319-25417-3_52.
[19] B.T. Nguyen, M.H. Trinh, T.V. Phan, H.D. Nguyen, Efficient real-time emotion detection using camera and facial landmarks, in: 2017 Seventh International Conference on Information Science and Technology (ICIST), IEEE, 2017, pp. 251–255, https://doi.org/10.1109/ICIST.2017.7926765.
[20] Ce Liu, J. Yuen, A. Torralba, SIFT flow: dense correspondence across scenes and its applications, IEEE Trans. Pattern. Anal. Mach. Intell. 33 (5) (2011) 978–994, https://doi.org/10.1109/TPAMI.2010.147.
[21] I. Michael Revina, W.R. Sam Emmanuel, Face expression recognition with the optimization-based Multi-SVNN classifier and the modified LDP features, J. Vis. Commun. Image Represent. 62 (2019) 43–55, https://doi.org/10.1016/j.jvcir.2019.04.013.
[22] R. Halder, S. Sengupta, A. Pal, S. Ghosh, D. Kundu, Real-time facial emotion recognition based on image processing and machine learning, Int. J. Comput. Appl. 139 (11) (2016) 16–19, https://doi.org/10.5120/ijca2016908707.
[23] F.Z. Salmam, A. Madani, M. Kissi, Emotion Recognition from Facial Expression Based on Fiducial Points Detection and using Neural Network, Int. J. Electr. Comput. Eng. (IJECE) 8 (1) (2018) 52, https://doi.org/10.11591/ijece.v8i1.pp52-59.
[24] P. Ekman, W.V. Friesen, Constants across cultures in the face and emotion, J. Pers. Soc. Psychol. 17 (2) (1971) 124–129, https://doi.org/10.1037/h0030377.
[25] A. Hassouneh, A.M. Mutawa, M. Murugappan, Development of a Real-Time Emotion Recognition System Using Facial Expressions and EEG based on machine learning and deep neural network methods, Inform. Med. Unlocked. 20 (2020) 100372, https://doi.org/10.1016/j.imu.2020.100372.
[26] A. Gupta, S. Arunachalam, R. Balakrishnan, Deep self-attention network for facial emotion recognition, Procedia Comput. Sci. 171 (2020) 1527–1534, https://doi.org/10.1016/j.procs.2020.04.163.
[27] Z. Peng, J. Li, Z. Sun, Emotion recognition using generative adversarial networks, in: 2020 International Conference on Computer Engineering and Intelligent Control (ICCEIC), IEEE, 2020, pp. 77–80, https://doi.org/10.1109/ICCEIC51584.2020.00023.
[28] S. Li, W. Deng, Blended Emotion in-the-Wild: multi-label facial expression recognition using crowdsourced annotations and deep locality feature learning, Int. J. Comput. Vis. 127 (6–7) (2019) 884–906, https://doi.org/10.1007/s11263-018-1131-1.
[29] U. Gogate, A. Parate, S. Sah, S. Narayanan, Real-time emotion recognition and gender classification, in: 2020 International Conference on Smart Innovations in Design, Environment, Management, Planning and Computing (ICSIDEMPC), IEEE, 2020, pp. 138–143, https://doi.org/10.1109/ICSIDEMPC49020.2020.9299633.
[30] S.A. Kumar, K.K. Thyaghrajan, Facial expression recognition with auto-illumination correction, in: 2013 International Conference on Green Computing, Communication and Conservation of Energy (ICGCE), Chennai, India, 2013, pp. 843–846, https://doi.org/10.1109/ICGCE.2013.6823551.
[31] S.D. Lalitha, K.K. Thyagharajan, Micro-facial expression recognition based on deep-rooted learning algorithm, Int. J. Comput. Intell. Syst. (2019) 903–913, https://doi.org/10.2991/ijcis.d.190801.001.
[32] K. Dinakaran, E. Ashokkrishna, Efficient regional multi feature similarity measure based emotion detection system in web portal using artificial neural network, Microprocess. Microsyst. 77 (2020) 103112, https://doi.org/10.1016/j.micpro.2020.103112.
[33] B. Fasel, J. Luettin, Automatic facial expression analysis: a survey, Pattern. Recognit. 36 (1) (2003) 259–275, https://doi.org/10.1016/S0031-3203(02)00052-3.

[34] Baristuzemenn Artificial Intelligence Engineer, CNN (Convolutional Neural Networks), 2024. https://medium.com/@baristuzemenn/cnn-convolutional-neural-networks-b589f7daf309.
[35] T.F. Cootes, C.J. Taylor, D.H. Cooper, J. Graham, Active shape models—Their training and application, Comput. Vision Image Understanding 61 (1) (1995) 38–59. https://www.sciencedirect.com/science/article/pii/S1077314295900603.
[36] Book: "Statistical Models in Computer Vision" by Tim Cootes and Christopher Taylor.
[37] G. González, F. Fleuret, P. Fua, Automated Delineation of Dendritic Networks in Noisy Image Stacks, in: D. Forsyth, P. Torr, A. Zisserman (Eds.), Computer Vision – ECCV 2008. ECCV 2008. Lecture Notes in Computer Science, Berlin, Heidelberg 5305, Springer, 2008, https://doi.org/10.1007/978-3-540-88693-8_16.
[38] Cootes, T.F., & Taylor, C.J. (2004). Statistical models of appearance for computer vision. https://www.researchgate.net/publication/2535636_Statistical_Models_of_Appearance_for_Computer_Vision.
[39] N. Ajlouni, A. Özyavaş, M. Takaoğlu, et al., Medical image diagnosis based on adaptive Hybrid Quantum CNN, BMC. Med. ImAging 23 (2023) 126, https://doi.org/10.1186/s12880-023-01084-5.
[40] U.C. Aytaç, A. Güneş, N. Ajlouni, A novel adaptive momentum method for medical image classification using convolutional neural network, BMC Med. Imaging 22 (2022) 34, https://doi.org/10.1186/s12880-022-00755-z.